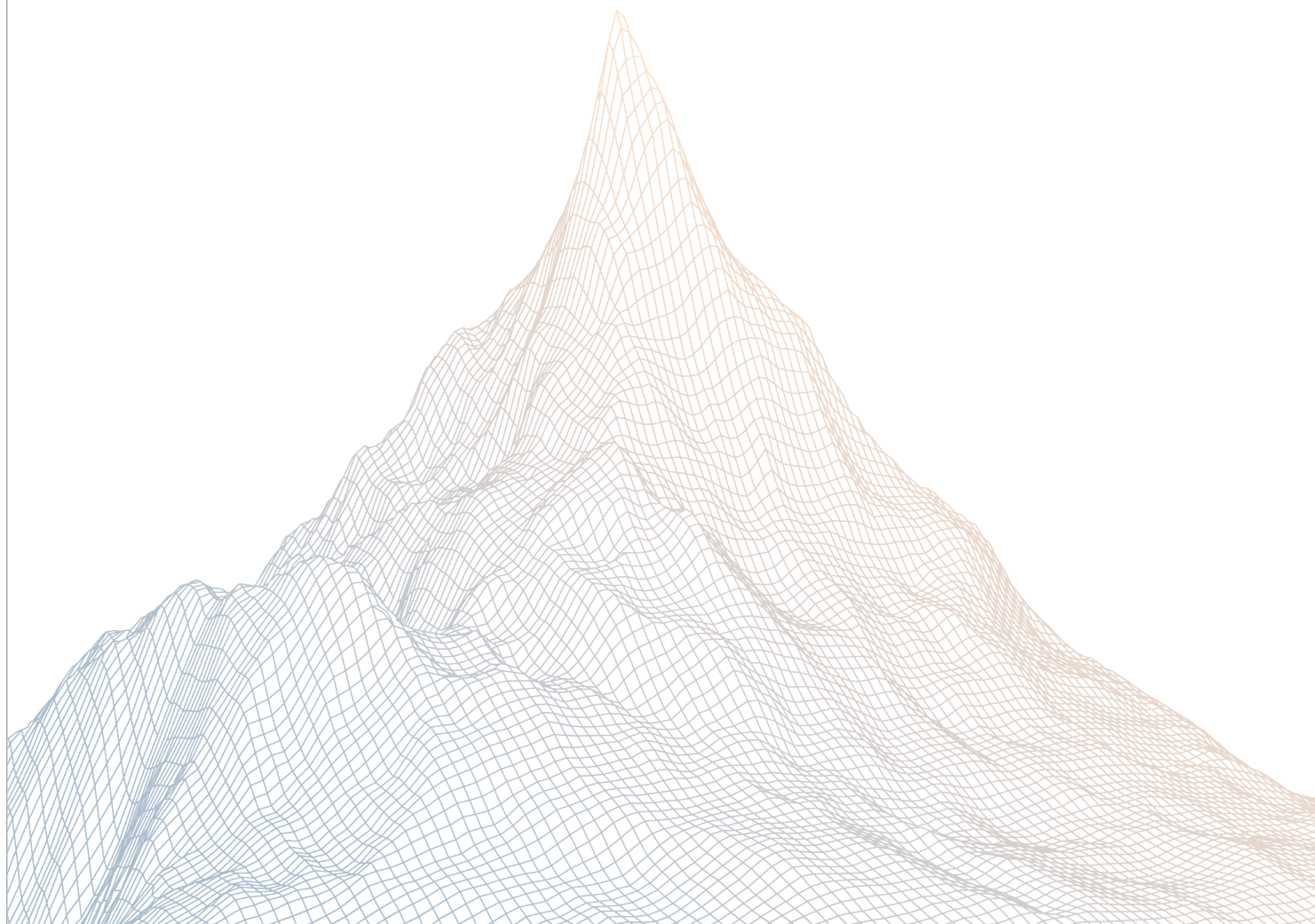


Bridge

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

August 21st to 25th, 2026

AUDITED BY:

cccZ
said

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Bridge	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Low Risk	7
4.2	Informational	11

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Bridge

An entirely new payments platform, built with stablecoins, to simplify global money movement.

2.2 Scope

The engagement involved a review of the following targets:

Target	erc20-stablecoin-template
---------------	---------------------------

Repository	https://github.com/withbridge/erc20-stablecoin-template
-------------------	---

Commit Hash	40638031289092232022a1332e857c293a41c8e8
--------------------	--

Files	src/**/*.sol
--------------	--------------

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/withbridge/erc20-stablecoin-template
-------------------	---

Commit Hash	3764e3bfc89434e61a0e5095edc0d211b8dcba9
--------------------	---

Files	src/**/*.sol
--------------	--------------

2.3 Audit Timeline

August 21, 2026	Audit start
August 25, 2026	Audit end
August 25, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	5
Total Issues	9

3

Findings Summary

ID	Description	Status
L-1	Replacing a handler leaves its reserve stores behind	Resolved
L-2	An expired approval can be revived	Acknowledged
L-3	A rate-limit change can block a published approval	Acknowledged
L-4	Published approvals are not scoped to the active stablecoin configuration	Acknowledged
I-1	Controller mint limits reject the exact configured maximum	Resolved
I-2	Reserve and stablecoin decimals are not converted	Resolved
I-3	Handler setter can also change per-transaction mint limits	Resolved
I-4	Canonical controller interfaces omit implemented actions	Resolved
I-5	Operation IDs are not unique across controller deployments	Resolved

4

Findings

4.1 Low Risk

A total of 4 low risk findings were identified.

[L-1] Replacing a handler leaves its reserve stores behind

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [TokenAuthority.sol#L304-L318](#)
- [ReserveLedgerBackedHandler.sol#L47-L118](#)
- [ReserveStore.sol#L15-L37](#)

Description:

`setTokenHandler()` accepts any ERC-165 responder and overwrites the active handler. A backed handler stores each stablecoin's reserve-store address in its own storage. The reserve store approves only the handler that created it.

After a handler change, the new handler has no entry for the asset. Its burn and unwrap routes revert because no reserve store exists in its mapping. The old collateral remains controlled through the old handler. There is no transition that verifies balances, moves collateral, or binds the replacement to the same `TokenAuthority` and strategy.

Recommendations:

Make handler replacement a paused, per-asset migration. Verify handler wiring, move or re-authorize collateral atomically, and verify the resulting balance.

Bridge: Resolved with [@a08499ff92 ...](#)

Zenith: Verified.

[L-2] An expired approval can be revived

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [MintIntent.sol#L139-L154](#)
- [MintIntent.sol#L202-L222](#)

Description:

`extendApproval()` accepts any new expiry later than the stored expiry. It does not require the existing expiry to be in the future. An approval is invalidated for consumption only while the current time is past its expiry; expiry does not set an irreversible flag.

A publisher can extend an expired approval and make the old operation and hold ID executable again. Off-chain systems that treat expiry as final can reconcile or cancel an operation and later see it minted.

Recommendations:

Require the existing expiry to be in the future before extension. After expiry, require a new approval with new operation and hold IDs.

Bridge: Acknowledged. We opt for trying to push mints through vs needing to reverse all the steps that happen before we get to this point.

[L-3] A rate-limit change can block a published approval

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [MintIntent.sol#L49-L95](#)
- [TokenAuthority.sol#L166-L178](#)
- [TIP20Controller.sol#L141-L153](#)

Description:

Publishing an approval does not reserve the current transaction limit or the executor allowance. `mintWithApproval()` reads both live values later. A rate-limit setter can lower either value after publication and before consumption, making a previously valid approval revert.

The state rolls back on that revert, but the publisher must restore capacity, extend, revoke, or replace the approval. This creates an avoidable failure point in the two-step workflow.

Recommendations:

Snapshot the required allowance and transaction-limit policy when publishing, or document that rate changes revoke pending approvals and provide a clear reissue path.

Bridge: Acknowledged. We would update the limit if we got to this point.

[L-4] Published approvals are not scoped to the active stablecoin configuration

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [MintIntent.sol#L49-L95](#)
- [TokenAuthority.sol#L304-L362](#)
- [TokenAuthority.sol#L166-L179](#)

Description:

`publishApproval()` accepts any nonzero stablecoin address. It does not require that the asset is registered, and it does not store the asset registration or handler configuration that exists when the approval is published.

An approval can be published before an asset is registered, then consumed after a later registration gives that address a handler and mint configuration. The same problem appears after `unregisterStablecoin()` and re-registration. `setTokenHandler()` can also change the handler after publication. The approval is bound only to the token address, but consumption uses the configuration that exists later.

Recommendations:

Require an existing registration when publishing. Store a per-asset configuration epoch in the approval and require it during consumption. Increment the epoch when an asset is unregistered or its handler changes. `TIP20Controller` needs an equivalent stablecoin registry and epoch.

Bridge: Acknowledged. Handlers are only meant to abstract away the differences between different implementations. As part of our process, the asset will be registered before the mint is created.

4.2 Informational

A total of 5 informational findings were identified.

[I-1] Controller mint limits reject the exact configured maximum

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [TokenAuthority.sol#L273-L294](#)
- [TokenAuthority.sol#L437-L439](#)
- [TIP20Controller.sol#L267-L287](#)
- [TIP20Controller.sol#L414-L417](#)

Description:

Both controllers define an absolute maximum of $1,000,000,000 * 1e6$. Their setter functions require a transaction limit and a minter allowance to be strictly below this value, but the actual mint path allows an amount equal to the maximum.

For example, a limit of $1,000,000,000$ token units can be minted, but the same value cannot be configured as the per-transaction limit or allowance.

This is a boundary inconsistency only.

Recommendations:

Use the same boundary everywhere. Either allow the exact maximum in the setters, or change the mint check and documentation to require a value strictly below it.

Bridge: Resolved with [@1c2f01ce38 ...](#)

Zenith: Verified.

[I-2] Reserve and stablecoin decimals are not converted

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ReserveLedgerWrappedHandler.sol#L40-L77](#)
- [ReserveLedgerBackedHandler.sol#L47-L60](#)
- [ReserveLedgerBackedHandler.sol#L86-L99](#)
- [StablecoinTemplateV3.sol#L44-L51](#)
- [StablecoinTemplateV3.sol#L124-L129](#)

Description:

The handlers move the same raw amount of reserve tokens and stablecoins. They do not convert between the two tokens' decimals, and the migration check compares raw balances directly.

For example, if the reserve uses 18 decimals and the stablecoin uses 6, depositing $1e6$ raw reserve units creates one nominal stablecoin but only 0.000000000001 nominal reserve tokens. The migration check still sees equal raw values and can accept the state as fully backed.

The default examples use equal decimals, so this requires a deployment with unequal decimals. Once such a configuration is enabled, nominal backing can be wrong.

Recommendations:

Either require the reserve and stablecoin to use the same decimals in every supported deployment, or apply an explicit conversion in minting, burning, wrapping, unwrapping, reserve stores, and migration checks.

Bridge: Resolved with [@70581e67e3 ...](#)

Zenith: Verified.

[I-3] Handler setter can also change per-transaction mint limits

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [TokenAuthority.sol#L273-L280](#)
- [TokenAuthority.sol#L327-L344](#)
- [TokenAuthority.sol#L353-L362](#)

Description:

MINT_RATE_LIMIT_SETTER_ROLE is the direct writer for mintTxnLimits through setTxnMintLimit. However, registerStablecoin is protected by TOKEN_AUTHORITY_HANDLER_SETTER_ROLE and also writes the same mapping. The handler setter can therefore choose a per-transaction mint limit when registering a new stablecoin. For an existing stablecoin, it can unregister it (which deletes the stored limit) and register it again with a different handler and limit.

This is a role-boundary mismatch: the handler setter controls more than handler routing, while the rate-limit setter is not the only role that controls the transaction cap.

Recommendations:

Separate stablecoin registration from rate-limit configuration. Have registerStablecoin set only the handler and require MINT_RATE_LIMIT_SETTER_ROLE to set the transaction limit.

Bridge: Resolved with [@70581e67e3 ...](#)

Zenith: Verified.

[I-4] Canonical controller interfaces omit implemented actions

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ITokenAuthority.sol](#)
- [TokenAuthority.sol](#)
- [ITIP20Controller.sol](#)
- [IMintIntent.sol](#)

Description:

The concrete `TokenAuthority` exposes public wrapping, stablecoin registration, `MintIntent` publication/revocation/extension/query, and mint-intent-version operations that are not all declared by `ITokenAuthority`. `TIP20Controller` exposes the inherited `MintIntent` lifecycle, but `ITIP20Controller` does not declare those functions. Typed integrations using the canonical interfaces therefore cannot compile calls to supported operations without switching interfaces or using raw selectors.

`IMintIntent` also declares an `ApprovalExpired` error that the implementation does not use; expired approval consumption returns `InvalidApprovalParams` instead. This can make client-side error handling miss the documented failure type.

Recommendations:

Make both `ITokenAuthority` and `ITIP20Controller` inherit `IMintIntent`, so the `MintIntent` publication, revocation, extension, and query functions are available through the canonical controller types. Also add the `TokenAuthority` functions currently missing from its interface: `wrap`, `registerStablecoin`, `unregisterStablecoin`, and `setMintIntentVersion`. Keep the controller-specific functions already implemented by `ITIP20Controller`, and verify that both interfaces include every public function intended for integrations.

Remove `ApprovalExpired` from `IMintIntent` if it is not part of the runtime behavior, or change approval consumption to return that error for an expired approval. The current implementation returns `InvalidApprovalParams` with the expiry flag set.

Bridge: Resolved with [@f02a2ccec3...](#)

Zenith: Verified.

[I-5] Operation IDs are not unique across controller deployments

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [MintIntentStorage.sol#L16-L38](#)
- [TokenAuthority.sol#L194-L205](#)
- [TIP20Controller.sol#L182-L193](#)

Description:

The mint-intent storage namespace fixes a storage slot, not a protocol-wide registry. Each `TokenAuthority` and `TIP20Controller` proxy keeps its own operation-ID and hold-ID mappings. The same IDs can be published and consumed once in every controller deployment.

If one source bridge operation is expected to be processed once across controller deployments, a publisher can publish it in both controllers and the local checks pass in both places. The code calls operation IDs globally unique, but enforces uniqueness only inside one proxy.

Recommendations:

Use one shared intent registry when source operations must be globally single-use. Otherwise, include and enforce a controller and destination-domain identifier in the source operation ID, and document that IDs are controller-scoped.

Bridge: Resolved with [@3764e3bfc...](#)

Zenith: Verified.